

Sparse Collaborative Filtering on MovieLens 100K: User and Item Neighbourhoods

Student: Ivan Tsarikov

Assignment: A03 - Collaborative Filtering (Week 3)

Team: Individual

Date: 2026-09-29

Abstract - Sparse movie-rating data make a missing user-movie preference hard to estimate from neighbours. We built user-based and item-based collaborative filtering on MovieLens 100K using only co-rated evidence, overlap shrinkage, a shared smoothed baseline, and clearly labelled fallbacks. On a frozen-parameter 10,000-pair confirmation split, user and item methods reached MAE 0.7299 and 0.7233, respectively, with more than 99% supported coverage for each. The item method had a small accuracy edge in this split, while sparse titles and cold starts still require explicit fallback handling.

Index Terms - collaborative filtering, sparse ratings, neighbourhood models, MovieLens, cold start

1. Introduction

The Week 3 task asks for two estimates of the same missing movie rating: one from similar users and one from similar movies. In the assignment's matrix convention, movies are rows and users are columns; an empty cell is unknown, not a zero rating. MovieLens 100K has 100,000 observed cells among 943 users and 1,682 movies, a density of 6.30% [1], [2].

The supplied Week 3 app trains a TensorFlow.js matrix-factorization model and shows one score, so it does not expose either required neighbourhood calculation. A user choosing an unseen title needs to see both estimates and their evidence. For example, the full-history app ranks *Casablanca* for User 1 at about 4.80/5 under both methods, yet their remaining Top-5 lists differ.

Contributions.

- Implemented both sparse, overlap-weighted CF orientations with the same fallback policy, explicit support labels, and separate unseen Top-5 lists.
- Compared rating error, support, latency, sparse cases, and one Week 2 content-based example using reproducible inputs and a frozen-parameter confirmation split.

2. Related Work

GroupLens documents the dataset's exact schema and provenance [1], [2]. Herlocker et al. describe user-neighbour CF and algorithmic choices for sparse common ratings [3]; Sarwar et al. develop item-based similarity and adjusted cosine [4]. Koren et al. explain matrix factorization as a latent-factor alternative [5]. Evaluation must separate error from coverage and task fit [6], while cold-start work distinguishes new items and new users from warm predictions [7].

Alternatives considered. Mean filling is easy to implement but inserts invented ratings into most matrix cells. Matrix factorization can fit latent factors on observed ratings but adds training cost and is less transparent for the requested similarity comparison [5]. We kept the neighbourhood methods as the deliverable and used factorization as an analytical comparator, without claiming an unrun MF benchmark.

3. Method

The browser keeps maps of ratings by user and by movie. Both methods compare only shared observations, require at least three, and multiply centred cosine by $n/(n+10)$, where n is the shared count. User similarity centres each person's co-rated scores by that person's training mean. Item similarity centres each co-rater's scores by that user's training mean, as adjusted cosine [3], [4].

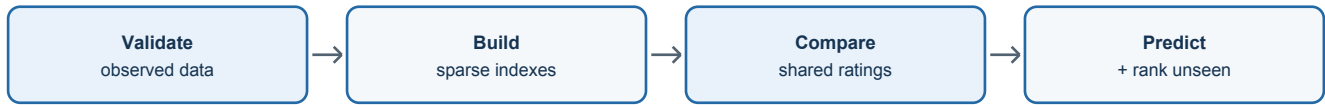


Figure 1. The same validated sparse ratings feed both similarity orientations and a shared prediction baseline.

3.1 Prediction and missing-value policy

A smoothed baseline uses the global training mean plus user and movie deviations. Positive similarities select at most 20 neighbours. The prediction adjusts the target baseline by their weighted residuals; fewer than two contributing neighbours produce a labelled baseline fallback. Predictions are clipped to the valid 1-5 range. No unknown matrix cell is imputed for similarity.

$$b(u,i) = \text{clip}(\mu + n(u)/(n(u)+10) [\text{mean}(u)-\mu] + n(i)/(n(i)+10) [\text{mean}(i)-\mu])$$

$$\text{pred}(u,i) = \text{clip}(b(u,i) + \text{sum}[s \times (\text{observed-neighbour rating} - \text{neighbour baseline})] / (\text{sum}[s] + 0.5))$$

For user CF, neighbours are users who rated target movie i ; for item CF, neighbours are movies already rated by u . The production app ranks supported candidates before fallbacks, excludes every known movie, and resolves remaining ties by estimated rating, support, and ID.

3.2 Configuration, tools, and controls

Component	Choice	Reason / control
Similarity	Shared-only centred cosine; $n \geq 3$; $n/(n+10)$	Do not fabricate agreement from absent ratings; discount tiny overlap
Aggregation	Positive Top-20; ≥ 2 contributors; $\gamma=0.5$	Require more than one vote and shrink weak residual changes
Baseline	User/item mean deviations; $\alpha=\beta=10$	Finite estimate even when no supported neighbours exist
Parser/UI	Plain browser JavaScript; UTF-8 strict decode	Reject malformed rows and unsafe title insertion; preserve starter layout
Evaluation	Python 3.14.3; fixed RNG seeds	Independent implementation; split before all means and similarities

The page uses no production library or TensorFlow.js dependency. Automated app checks use Node.js v26.9.0; the evaluation is an independent Python standard-library implementation. The exploratory 80/10/10 split used seed 20260929. Its validation grid selected the two-neighbour floor and $\gamma=0.5$; its test outcomes were also seen during development, so they are not used as final evidence. Settings were frozen before a second 80/10/10 resplit at seed 20260930.

4. Experiments

Protocol. The 100,000 MovieLens ratings were shuffled with seed 20260930 into 80,000 training, 10,000 validation, and 10,000 test ratings, keeping every test user and movie present in training. Both methods scored the same held-out pairs. Means and similarities used training rows only. This second split is a frozen-parameter robustness check on the same historical dataset, not a separate population or an online outcome. Local evaluation ran on macOS 26.4/arm64 in Python 3.14.3; timing includes Python and cache effects.

Confirmation test (n=10,000)	User CF	Item CF
MAE, all pairs (lower better)	0.7299	0.7233
RMSE, all pairs	0.9337	0.9207
Supported pairs	9,945 (99.45%)	9,920 (99.20%)
MAE on common supported 9,911	0.7281	0.7217
Mean Python query, ms	0.230	0.307

Table 1. Confirmation ratings are identical for the two methods. All-pair errors include labelled fallback predictions; the shared baseline was not benchmarked separately.

Item CF's MAE is lower by 0.0066 on this split. A paired bootstrap 95% interval for user-minus-item MAE is [0.0019, 0.0118], conditional on these split pairs; it does not measure variability across independent datasets. On the 9,911 pairs supported by both, item CF also has lower MAE (Table 1), so the all-pair difference is not explained solely by unequal fallback rates.

Top-5 check: 100 sampled users	User CF	Item CF
Users with ≥ 1 held-out positive hit	24/100	24/100
Held-out positives retrieved	29/700	28/700
Mean full Top-5 query, seconds	0.039	0.321

Table 2. A positive is a held-out rating ≥ 4 ; users were sampled by a fixed seed from those with at least one positive. Unknown unrated movies were not treated as negatives. These sparse hits do not estimate watch-time or business impact.

Exploratory validation only	Joint mean MAE	User Hit@5	Item Hit@5
One contributor; gamma=0	0.732881	5/100	12/100
Two contributors; gamma=0.5	0.731367	19/100	26/100

Table 3. An unshrunk residual from one contributor could drive a Top-5 estimate to 5.0. The contributor floor and gamma reduced this validation failure. The first split's test values were seen during development and are excluded from final claims.

4.1 Critical checks and hand calculation

For a five-movie toy matrix, Users 1 and 2 share four ratings. Their mean-centred shared vectors are [2, 1, -2, -1] and [2, 1, -2, -2]: dot product 11, squared norms 10 and 13. Thus user similarity = $11/\sqrt{10 \times 13} \times 4/(4+10) = 0.275647$. The adjusted item similarity of Movies 5 and 1 has dot 4, squared norms 2 and 9, and three shared users: $4/\sqrt{2 \times 9} \times 3/13 = 0.217571$. Both match the evaluator self-test and are printed so they can be checked without trusting a test label.

Production JavaScript and the independent Python evaluator matched on confirmation User 1 / Movie 138: user estimate 2.0682139721, item estimate 2.7421452382, baseline 3.0708231595. The Node suite passed seven tests covering parser errors, both similarity orientations, sparse support, cold starts, bounds, and unseen Top-5 lists on the complete files.

5. Discussion

5.1 User versus item: quality and computation

The observed item MAE advantage is small and specific to this warm-start confirmation split. Its support is slightly lower and its Python per-pair query slower; the production browser precomputes item-pair statistics, so these Python times are not browser latency. The full user space has 444,153 distinct pairs, versus 1,413,721 item pairs (3.18 times as many). The usual claim that item CF has fewer entities does not fit this dataset. Production cost also depends on sparse overlap, reuse, caching, and how often ratings update.

5.2 Missing-rating strategy trade-off

Strategy	Simplicity / cost	Main bias or limit
Mean fill	Easy formula; turns sparse cells into dense work	Unobserved cells can create artificial agreement
Co-rated count weighting (used)	Sparse intersections, neighbour search, and cache	Rare titles have fewer supported estimates
Matrix factorization [5]	Train factors and tune regularization; predict cheaply	Latent results are less transparent; new IDs need a fallback

Table 4. MF was inspected in the starter and literature; no MF error or runtime was measured in this study. A baseline returned at prediction time is not a filled matrix cell.

A two-user check shows the imputation risk. A rates $[X=5, Y=1, Z=?]$ and B rates $[X=5, Y=?, Z=1]$; both observed means equal 3. Mean filling gives centred vectors $[2, -2, 0]$ and $[2, 0, -2]$, whose cosine is $4/(\sqrt{8} \times \sqrt{8}) = 0.5$. Yet X is their only genuinely common rating: our $n \geq 3$ rule correctly marks their similarity unsupported. This is a constructed demonstration of mechanism, not a MovieLens accuracy result.

5.3 Cold start and little shared evidence

The full file has 141 one-rating movies; 63,490 of 444,153 distinct user pairs (14.29%) share at most two ratings. In the confirmation test, all 27 cases whose movie had one training rating fell back under both methods. Among 83 cases with two to four training ratings, support was 73.5% for user CF and 43.4% for item CF. Those small bands are descriptive, not proof of a universal error trend. A new user provides neither user overlap nor rated movies to aggregate; a new movie supplies neither neighbour ratings nor item overlap. Synthetic zero-history tests return the shared smoothed baseline with a fallback label.

5.4 Week 2 genres versus Week 3 ratings

For User 1, Week 2 profiles the three earliest positively rated movies: Monty Python and the Holy Grail, The Empire Strikes Back, and Jean de Florette. Week 3 uses all 272 observed ratings. All three lists exclude the same 272 rated titles; this is a descriptive comparison of different inputs, not a controlled quality test.

#	Week 2: genre cosine	Week 3: user CF	Week 3: item CF
1	Crying Game, The (1992) #631	Casablanca (1942) #483	Casablanca (1942) #483
2	First Knight (1995) #720	Close Shave, A (1995) #408	Schindler's List (1993) #318
3	Casablanca (1942) #483	Pather Panchali (1955) #1449	Manchurian Candidate, The (1962) #657
4	Titanic (1997) #313	Glory (1989) #651	Close Shave, A (1995) #408
5	Boat, Das (1981) #515	Secrets & Lies (1996) #285	Third Man, The (1949) #513

Table 5. Full-data illustration, not a held-out performance table. Only Casablanca is shared by Week 2 and either CF list; the two CF lists share two titles. Genre tags retrieve thematic matches, while neighbours draw on other viewers' rating patterns.

5. Discussion (continued)

5.5 A failure case, root cause, and repair

Observed failure	Root cause	Repair and check
Movie 543's title displayed a replacement character in place of its accented e.	The supplied u.item contains legacy ISO-8859-1 bytes, but the starter reads it as UTF-8 (data.js lines 13-15).	Transcoded only the working u.item to valid UTF-8; exact Latin-1 to UTF-8 equality and a real-data test now confirm the title. u.data remains byte-identical.

The broader starter-to-task gap was also checked: its embedding dot product supplies one rating but neither required user/item similarity. The repaired app calculates both without waiting for browser training. The encoding failure is the specific observed defect above; an unconstrained out-of-range dot-product prediction was only a risk, not a claimed observation.

5.6 What worked and what did not

- Sparse indexes, minimum overlap, and residual shrinkage kept supported predictions separate from baseline fallbacks; the same rules work in both orientations.
- The confirmation split preserved the small item-CF MAE advantage, but Top-5 hit users tied at 24/100 and item CF used a larger pair space.
- Rare-movie support remains fragile. For held-out User 1 / D3: The Mighty Ducks, actual rating 1, user CF predicted 2.068 and item CF 2.742; both overpredicted.

Next improvement. Evaluate a content-backed cold-start path for movies with little rating history, while preserving the support label. Test it on a genuinely disjoint new-item holdout rather than assuming that better offline rating error produces retention or revenue.

6. AI Usage Disclosure

Tool and scope. OpenAI Codex (GPT-6 family; exact service build not exposed) and delegated Codex agents were used for source inspection, code and test generation, debugging, primary-reference checks, quantitative evaluation, documentation, and PDF drafting. No additional AI tool is claimed here.

Machine-executed checks. Codex ran the Node tests, a Python hand-case self-test, two fixed-seed evaluations, byte and encoding comparisons, and the JS/Python parity check reported in Section 4. The derivation printed in Section 4 is reproducible line by line. These are agent-executed checks, not a claim that Ivan personally performed them.

Personal verification and limits. No independent personal verification by Ivan is asserted in this draft. Before submission he should recalculate the toy cosine, rerun the evaluation command, and inspect the data, findings, and PDF. The 10,000 test errors were aggregated by the evaluator; neither every prediction nor the cited papers' full texts were independently checked by the student in this record. The required unmodified, tool-native session.json is a separate audit file and is not embedded in this PDF.

Five-loop reflection. BUILD: implement both CF branches. ASK WHY: contrast co-rated-only weights with mean fill and factorization. TEST: compute a similarity by hand and compare JS with Python. EXPLAIN: separate error, support, and cost. IMPROVE: target sparse-item cold start with a disjoint holdout.

Reproducibility. Headline metrics come from a frozen-parameter confirmation run. A prior exploratory run records parameter selection and development, including a viewed first-split test. The Week 2 comparison is a full-history illustration. The generator checks the recorded run protocol and source counts before building this report.

References

- [1] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," *ACM Trans. Interactive Intelligent Systems*, vol. 5, no. 4, Art. 19, pp. 1-19, Dec. 2015. doi: 10.1145/2827872. [Online]. Available: <https://doi.org/10.1145/2827872>. [Accessed: Sep. 29, 2026].
- [2] GroupLens Research, "MovieLens 100K dataset README," University of Minnesota, n.d. [Online]. Available: <https://files.grouplens.org/datasets/movielens/ml-100k/README>. [Accessed: Sep. 29, 2026].
- [3] J. L. Herlocker, J. A. Konstan, A. Borchers, and J. Riedl, "An algorithmic framework for performing collaborative filtering," in *Proc. SIGIR*, 1999, pp. 230-237. doi: 10.1145/312624.312682. [Online]. Available: <https://doi.org/10.1145/312624.312682>. [Accessed: Sep. 29, 2026].
- [4] B. Sarwar, G. Karypis, J. A. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proc. WWW*, 2001, pp. 285-295. doi: 10.1145/371920.372071. [Online]. Available: https://files.grouplens.org/papers/www10_sarwar.pdf. [Accessed: Sep. 29, 2026].
- [5] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30-37, Aug. 2009. doi: 10.1109/MC.2009.263. [Online]. Available: <https://doi.org/10.1109/MC.2009.263>. [Accessed: Sep. 29, 2026].
- [6] J. L. Herlocker, J. A. Konstan, L. G. Terveen, and J. T. Riedl, "Evaluating collaborative filtering recommender systems," *ACM Trans. Inf. Syst.*, vol. 22, no. 1, pp. 5-53, Jan. 2004. doi: 10.1145/963770.963772. [Online]. Available: <https://grouplens.org/site-content/uploads/evaluating-TOIS-20041.pdf>. [Accessed: Sep. 29, 2026].
- [7] A. I. Schein, A. Popescul, L. H. Ungar, and D. M. Pennock, "Methods and metrics for cold-start recommendations," in *Proc. SIGIR*, 2002, pp. 253-260. doi: 10.1145/564376.564421. [Online]. Available: <https://dpennock.com/papers/schein-sigir-2002-cold-start.pdf>. [Accessed: Sep. 29, 2026].